

Making `std::istream::ignore` less surprising

- - [Revision History](#)
 - [Introduction](#)
 - [Discussion](#)
 - [Detailed discussion](#)
 - [Writing correct code is unnecessarily hard](#)
 - [Possible solutions](#)
 - [Change ignore to handle negative delim values](#)
 - [Split ignore into two functions and change delim to char_type](#)
 - [Add an overload of ignore that takes a char_type](#)
 - [As above, but constrain the new overload to prevent ambiguities](#)
 - [Do nothing](#)
 - [Proposed wording](#)
 - [Acknowledgements](#)
 - [References](#)

Revision History

Changes in R1:

- Extend discussion to avoid implicit assumptions about `to_int_type` details.
- Adjust proposed wording to add *Constraints*: to the new overload.
- Adjust title to be specific to `std::istream` not `std::basic_istream`.

Introduction

`std::istream::ignore(n, delim)` has surprising behaviour if `delim` is a `char` with a negative value. We can remove the surprise and make code more robust.

Discussion

Passing a `char` to `std::istream::ignore` as the delimiter is a bug. The delimiter argument should be an `int_type` value, so correct code must do `is.ignore(n, std::char_traits<char>::to_int_type(delim))` instead. There is no guarantee that an implicit conversion from a `char` value to `int_type` yields the same value as calling `to_int_type`. On platforms where `char` is signed, if the value of `EOF` fits in `char` then there is always at least one `char` value (the one equal to `EOF`) that cannot be used as a delimiter to `std::istream::ignore`, because `c == to_int_type(c)` is false.

For example, on a platform where `char` is signed, `EOF` equals `-1`, and the literal encoding is ISO-8859-1 or Windows-1252, the call `is.ignore(n, 'ÿ')` will never match the delimiter, because `'ÿ' == EOF` is true.

In theory, the `to_int_type` mapping could be `~c` or `c + 256`, where the former means that most `char` values will match unintended characters, and the latter means that no `char` value passed directly to `ignore` will ever match.

In practice, real implementations use a more straightforward mapping, such that calling `to_int_type(c)` is equivalent to `(int)(unsigned char)c` and `istream` functions that take an `int_type` argument expect values between -1 and 255. This matches how the C functions like `isalpha(int)` and `toupper(int)` work. So for all known implementations, an implicit conversion from `char` to `int_type` is incorrect for any negative `char` value.

Detailed discussion

This section assumes an implementation where `to_int_type` is equivalent to casting to `unsigned char`, but even for a hypothetical implementation where that isn't true, `is.ignore(n, ch)` is still wrong in general, as shown above.

A `delim` value passed to `std::istream::ignore` is matched using `traits_type::eq_int_type(rdbuf()->sgetc(), delim)`. The `sgetc()` function never returns negative values (except at EOF). It extracts a character `c` from the input sequence and then calls `traits_type::to_int_type(c)` to convert it to `int_type`, which produces a non-negative value. So if `delim` is negative, then the `eq_int_type` comparison always fails.

The `std::char_traits<char>::to_int_type(char c)` function converts the character `c` to a non-negative integer, as if by `(int)(unsigned char)c`. This allows the value `(int_type)-1` to be reserved for EOF without worrying about whether `char` is signed and whether `(char)-1` can equal `EOF`. But it means that any code dealing with raw `char` values from the stream must consistently use `to_int_type` to convert the (possibly negative) `char` value into a non-negative `int_type` so that all characters are represented in the same form and can be compared like-for-like.

So if a negative delimiter `char` can never match, this means that users should call `std::cin.ignore(n, std::char_traits<char>::to_int_type(c))` or `std::cin.ignore(n, (unsigned char)c)` in case `char` is signed on their platform, and `c` happens to have the most significant bit set, i.e., is a negative value. For example, on a typical x86_64 Linux system where `char` is signed, `std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\x80')` will always discard all input up to EOF, even if `\x80` is present in the input.

Writing correct code is unnecessarily hard

In generic code that works with streams of either `char` or `wchar_t` you need to use the more verbose `to_int_type` form rather than casting to `unsigned char`, because casting a `wchar_t` to `unsigned char` would be wrong. That's unfortunate, because `std::char_traits<wchar_t>::to_int_type` doesn't have the same "cast to unsigned" behaviour, and so passing a `wchar_t` directly to `std::wistream::ignore` without conversion works correctly. But to work with both `char` and `wchar_t`, generic code must assume the worst and defend against the signed `char` trap.

Even in non-generic code that only works with `char`, you still need to remember that the trap exists, and remember to avoid it. It's rare that I see anybody get that right for `std::isalpha(c)` et al, and I wasn't even aware of the need to do it for `ignore` until this week.

I suspect that most users are not aware of the need to use `to_int_type` here, which means that the `ignore` function is surprisingly fragile. It's also quite ugly to have to cast or deal with the traits type directly in a high-level `istream` API like `ignore`, which is not a low-level `streambuf` member function. Several `basic_streambuf` members use `int_type` for arguments and return values, but at the `basic_istream` level the other unformatted input functions that take a delimiter (`get` and `getline`) take a `char_type`. That means they are agnostic to whether `char` is signed or unsigned, and any conversion to `int_type` is done by the stream, not expected to be done by the caller. Arguments of type `int_type`

Possible solutions

Change ignore to handle negative delim values

We could modify the spec for `basic_istream::ignore` so that negative values (except for -1 which must be reserved for EOF) are automatically fed through `to_int_type` to "clean" them, so that they're in the same domain as the values returned by `sgetc()`.

The GNU C library takes this approach for its `<ctype.h>` functions, which are specified to take `int`, but which have undefined behaviour if the argument is not an `unsigned char` value or EOF. So `isalpha('\x80')` has undefined behaviour if `char` is signed. But with Glibc, it works. Values in the range [-128,-1) are handled as if converted to `unsigned char` automatically, so that negative `char` values don't produce undefined behaviour. Obviously it's still possible to misuse those functions, e.g., by passing an `int` value outside the range of `char` or `unsigned char`, e.g., `isalpha(1000)`. But the apparently simple `isalpha(c)` for a `char` value isn't undefined just because `c` happens to be a negative value of a signed type.

Taking the same approach for `ignore` would remove the trap for cases like `cin.ignore(n, '\x80')`, making it behave as `cin.ignore(n, std::char_traits<char>::to_int_type('\x80'))`. However, it would also "fix" cases like `cin.ignore(n, -10)` which are less likely to be correct. There would be no way to tell the difference between a `char` with the value -10 and an `int` with the value -10, but the latter seems odd, and possibly a bug. Depending how we specified it, this solution might also give defined behaviour to `cin.ignore(n, +1000)` and `cin.ignore(n, -9999)` which are just nonsense.

The other downside of this solution is that it only fixes negative delimiters less than or equal to -2, because -1 still has to be reserved to mean EOF. So on a platform where `char` is signed, `cin.ignore(n, '\xfe')` would work, but `cin.ignore(n, '\xff')` would not, because that value is `traits_type::eof()`. On a platform where `char` is unsigned, both work. So this solution removes *most* surprises, but not all, and doesn't have portable guarantees. Users should really still use `in.ignore(n, std::char_traits<char>::to_int_type(c))` to work with arbitrary delimiter characters on arbitrary platforms.

Split ignore into two functions and change delim to char_type

We could change `delim` to `char_type` and make `ignore` convert that to `int_type` internally by using `to_int_type`.

```
basic_istream& ignore(streamsize n = 1, int_type = traits_type::eof());  
basic_istream& ignore(streamsize n, char_type delim);
```

This would preserve the same behaviour for `in.ignore(n)`, i.e., ignore up to `n` characters or up to EOF, whichever happens first. But it would break explicit uses of `eof` as the second argument, e.g., `in.ignore(n, traits_type::eof())`. This would implicitly convert the `eof()` value to `char_type` and treat it as a delimiter. If `(char)-1` happens to be present in the next `n` characters of the input sequence, it would match and we would stop ignoring too soon. This would break too much code, and we can do better.

Add an overload of ignore that takes a char_type

We don't need to change the existing `ignore`, we can just add an overload that does the correct conversion to `int_type`:

```
basic_istream& ignore(streamsize n, char_type delim)  
{ return ignore(n, traits_type::to_int_type(delim)); }
```

This does exactly what users expect when calling `ignore(n, c)` with a `char_type` argument (even `(char)-1`), with no alarms and no surprises. The behaviour is entirely consistent for signed or unsigned `char` and always matches the given `delim` if it occurs in the input sequence.

The downside of this solution is that calls that pass a delimiter that is neither `int_type` nor `char_type` become ambiguous, e.g. `std::cin.ignore(n, 1ULL)` is valid today but would become ambiguous with this new overload. Arguably, that's a good thing. What is this code trying to do? What if it passes a value that doesn't even fit in `int_type`, e.g. `numeric_limits<int_type>::max()+1LL`? Maybe it's good for such calls to not compile.

Since the problem only exists for `char` istreams and not `wchar_t`, this new overload could be specified as present only when `char_type` is `char`. That would avoid introducing any ambiguities for `std::wistream`.

As above, but constrain the new overload to prevent ambiguities

```
basic_istream& ignore(streamsize n, same_as<char_type> auto delim)
{ return ignore(n, traits_type::to_int_type(delim)); }
```

This will only be selected by overload resolution when called with an argument of type `char_type`. For all other argument types, the existing overload will be selected and if the argument isn't of type `int_type` it will implicitly convert to `int_type`, exactly as happens today.

This doesn't change the meaning of any existing code, except for calls with negative `char_type` values which would start to work as users probably expected them to all along. The downside is the additional complexity of using a constrained overload, which would need to be emulated with SFINAE if vendors wanted to backport this fix to older standards modes.

Personally, I think the non-constrained overload is the best option, and that the cases which become ambiguous should probably be fixed to clarify what they're intending to do. Making the conversion to `int_type` explicit (and using `to_int_type` if appropriate) would probably be an improvement.

Do nothing

Boo! Users deserve better. Well, most of them. Some don't.

Proposed wording

The edits are shown relative to [N4981](#).

Modify the class synopsis in [istream.general] as shown:

```
// [istream.unformatted], unformatted input
streamsize gcount() const;
int_type get();
basic_istream& get(char_type& c);
basic_istream& get(char_type* s, streamsize n);
basic_istream& get(char_type* s, streamsize n, char_type delim);
basic_istream& get(basic_streambuf<char_type, traits>& sb);
basic_istream& get(basic_streambuf<char_type, traits>& sb, char_type delim);

basic_istream& getline(char_type* s, streamsize n);
basic_istream& getline(char_type* s, streamsize n, char_type delim);

basic_istream& ignore(streamsize n = 1, int_type delim = traits::eof());
basic_istream& ignore(streamsize n, char_type delim);
int_type peek();
basic_istream& read(char_type* s, streamsize n);
streamsize readsome(char_type* s, streamsize n);
```

Modify [istream.unformatted] as shown:

```
basic_istream& ignore(streamsize n = 1, int_type delim = traits::eof());
```

-25- *Effects*: Behaves as an unformatted input function (as described above). After constructing a sentry object, extracts characters and discards them. Characters are extracted until any of the following occurs:

- (25.1) — `n != numeric_limits<streamsize>::max()` ([numeric.limits]) and `n` characters have been extracted so far
- (25.2) — end-of-file occurs on the input sequence (in which case the function calls `setstate(eofbit)`, which may throw `ios_base::failure` ([iostate.flags]));
- (25.3) — `traits::eq_int_type(traits::to_int_type(c), delim)` for the next available input character `c` (in which case `c` is extracted).

[*Note 1*: The last condition will never occur if `traits::eq_int_type(delim, traits::eof())`. — *end note*]

-26- *Returns*: `*this`.

`basic_istream& ignore(streamsize n = 1, char_type delim);`

-?- *Constraints*: `is same v<char_type, char> is true`.

-?- *Effects*: Equivalent to: `return ignore(n, traits_type::to_int_type(delim));`

Acknowledgements

Thanks to Iain Sandoe and Ulrich Drepper for inspiring this. Thanks to Tom Honermann for getting me to state the problem in the general case.

References

[N4981](#), Working Draft - Programming Languages -- C++, Thomas Köppe, 2024.